



南開大學
Nankai University

计算机学院
计算机系统设计实验报告

PA1 实验报告

姓名：王一诺
学号：2312331
专业：计算机科学与技术

2026 年 4 月 26 日

目录

1 PA1 实验概述	2
1.1 实验目标	2
1.2 实验内容	2
2 PA1-1	2
2.1 实现正确寄存器结构体	2
2.2 实现单步执行、打印寄存器、扫描内存	3
2.2.1 单步执行 si	4
2.2.2 打印寄存器 info	5
2.2.3 扫描内存 x	6
3 PA1-2	8
3.1 表达式求值	8
4 PA1-3	17
4.1 监视点和断点	17
5 思考题与必答题	25
5.1 思考题	25
5.2 必答题	27

1 PA1 实验概述

1.1 实验目标

初步理解计算机系统的基本工作方式，认识“程序如何在机器上运行”这一核心问题，并通过 NEMU 这一简化的 x86 模拟器建立对硬件、操作系统与程序执行关系的直观认识。通过本次实验，学生需要熟悉 NEMU 的基本使用方法，理解调试器的基本交互流程，并在此基础上完成若干基础功能的实现，为后续 PA 的学习打下基础。

1.2 实验内容

本次 PA1 主要围绕 NEMU 的基本调试功能展开，内容大致包括三个部分：

- 实现寄存器结构，以及单步执行、查看寄存器信息、内存扫描这些调试器基本功能。
- 实现调试器中的表达式求值功能。
- 实现监视点和断点等基础调试机制。

实验过程中需要逐步完善调试器的命令支持，使其能够执行指令、查看 CPU 状态、读取内存内容、计算表达式并监控程序运行状态，从而初步具备一个可用的交互式调试环境。

2 PA1-1

2.1 实现正确寄存器结构体

CPU_state 结构体的设计目标是把模拟器中的 CPU 寄存器状态组织成一种既便于按位宽访问、又便于按寄存器名访问的数据结构，从而满足后续指令执行、调试输出和表达式求值等多个模块的共同需求。在 nemu/include/cpu/reg.h 中：

Listing 1: CPU_state

```
1 typedef struct {
2     union {
3         GPR gpr[8];
4
5         /* Do NOT change the order of the GPRs' definitions. */
6
7         /* In NEMU, rtlreg_t is exactly uint32_t. This makes RTL instructions
8          * in PA2 able to directly access these registers.
9          */
10        struct{
11            rtlreg_t eax, ecx, edx, ebx, esp, ebp, esi, edi;
12        };
13    };
14
15    vaddr_t eip;
16
17    EFLAGS eflags;
18 } CPU_state;
```

19

这里使用了联合体 union 来实现同一块存储空间的多重解释方式：外层的 gpr[8] 让 8 个通用寄存器以数组形式统一管理，每个数组元素内部又通过一个更小的联合体同时提供 32 位、16 位和 8 位两种低位访问形式，其中 _32 对应完整的 32 位寄存器值，_16 对应低 16 位，而 _8[2] 则把低 16 位再拆成两个字节，便于模拟 x86 中对 al/ah 之类低位寄存器的访问。与此同时，在同一个外层联合体中又定义了一个结构体，把 eax、ecx、edx、ebx、esp、ebp、esi、edi 这些寄存器按固定顺序命名出来，这样代码既可以用数组下标批量访问，也可以直接用寄存器名访问，避免了重复维护两套数据。结构体最后增加了 eip，用于保存当前指令地址，使程序计数器与通用寄存器统一纳入 CPU 状态管理中。

运行结果如下：

```
wyn@ubuntu:~/Documents/ics2017/nemu$ make run
+ CC src/device/timer.c
+ CC src/device/vga.c
+ CC src/device/device.c
+ CC src/device/io/mmio.c
+ CC src/device/io/port-io.c
+ CC src/device/keyboard.c
+ CC src/device/serial.c
+ CC src/memory/memory.c
+ CC src/monitor/debug/watchpoint.c
+ CC src/monitor/debug/expr.c
+ CC src/monitor/debug/ui.c
+ CC src/monitor/monitor.c
+ CC src/monitor/cpu-exec.c
+ CC src/monitor/diff-test/gdb-host.c
+ CC src/monitor/diff-test/diff-test.c
+ CC src/monitor/diff-test/protocol.c
+ CC src/misc/logo.c
+ CC src/cpu/exec/system.c
+ CC src/cpu/exec/arith.c
+ CC src/cpu/exec/logic.c
+ CC src/cpu/exec/data-mov.c
+ CC src/cpu/exec/control.c
+ CC src/cpu/exec/exec.c
+ CC src/cpu/exec/prefix.c
+ CC src/cpu/exec/special.c
+ CC src/cpu/exec/cc.c
+ CC src/cpu/decode/decode.c
+ CC src/cpu/decode/modrm.c
+ CC src/cpu/reg.c
+ CC src/cpu/intr.c
+ CC src/main.c
+ LD build/nemu
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 01:46:22, Apr 6 2026
For help, type "help"
(nemu) q
wyn@ubuntu:~/Documents/ics2017/nemu$
```

2.2 实现单步执行、打印寄存器、扫描内存

每条指令都会有对用户输入参数进行预处理的操作，所以可以提出来写一个公共函数：

Listing 2: trim 函数

```
1 static char *trim(char *str) {
2     if (str == NULL) {
3         return NULL;
4     }
5     while (*str == ' ' || *str == '\t') {
6         str++;
7     }
8     char *end = str + strlen(str);
```

```

9   while (end > str && (end[-1] == ' ' || end[-1] == '\t'))
10   { end--; }
11   *end = '\0';
12   return str;
13 }
14

```

trim 函数的作用是对用户输入的命令参数进行预处理，去除字符串首尾多余的空格和制表符，从而保证后续命令解析的准确性与稳定性。函数首先判断输入指针是否为空，如果为空则直接返回 NULL，避免对空指针进行解引用导致程序异常。接着，使用一个前移循环跳过字符串开头的空白字符，使指针直接指向第一个有效字符；随后通过 strlen 找到字符串末尾的位置，再利用从后向前回退的方式继续删除尾部的空格和制表符。最后，函数在新的字符串结束位置写入字符串终止符，从而在原字符串上实现就地截断。

此外，在 nemu/src/monitor/debug/ui.c 中的 cmd_table 命令表中添加这三条新指令的描述：

Listing 3: cmd_table

```

1   static struct {
2       char *name;
3       char *description;
4       int (*handler) (char *);
5   } cmd_table [] = {
6       { "help", "Display informations about all supported commands", cmd_help },
7       { "c", "Continue the execution of the program", cmd_c },
8       { "q", "Exit NEMU", cmd_q },
9       /* TODO: Add more commands */
10      { "si", "Step through N instructions (default is 1)", cmd_si },
11      { "info", "Print information about registers or watchpoints", cmd_info },
12      { "x", "Examine memory: x N EXPR", cmd_x },
13  };
14

```

2.2.1 单步执行 si

Listing 4: cmd_si

```

1   static int cmd_si(char *args) {
2       int steps = 1;
3       if (args != NULL) {
4           steps = atoi(args);
5       }
6       cpu_exec(steps);
7       return 0;
8   }
9

```

cmd_si 函数实现了单步执行功能，是调试器中最基础的控制命令之一。思路非常简单：先根据用户输入确定要执行的指令条数，然后调用底层的 cpu_exec 函数推进模拟器运行。函数一开始默认将步数

设为 1，如果用户直接输入 si 而不带参数，程序就只执行一条指令后停下；如果用户提供了参数，则通过 atoi 将参数字符串转换为整数，作为实际执行步数。随后，函数调用 cpu_exec(steps)，由 CPU 执行模块完成指定数量的指令执行。

运行结果如下：

```
+ CC src/cpu/intr.c
+ CC src/main.c
+ LD build/nemu
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 00:54:50, Apr  7 2026
For help, type "help"
(nemu) si
100000: b8 34 12 00 00      movl $0x1234,%eax
(nemu) si
100005: b9 27 00 10 00      movl $0x100027,%ecx
(nemu) si
10000a: 89 01              movl %eax,%ecx
(nemu) si
10000c: 66 c7 41 04 01 00   movw $0x1,0x4(%ecx)
(nemu) si
100012: bb 02 00 00 00      movl $0x2,%ebx
(nemu) si
100017: 66 c7 84 99 00 e0 ff 01 00   movw $0x1,-0x2000(%ecx,%ebx,4)
(nemu) si
100021: b8 00 00 00 00      movl $0x0,%eax
(nemu) so
Unknown command 'so'
(nemu) si
nemu: HIT GOOD TRAP at eip = 0x00100026
100026: d6                nemu trap (eax = 0)
(nemu) q
```

2.2.2 打印寄存器 info

Listing 5: cmd_info

```
1 static int cmd_info(char *args) {
2     args = trim(args);
3     if (args == NULL || args[0] == '\0') {
4         printf("Usage: info r (to print registers) or info w (to print watchpoints)\n");
5         return 0;
6     }
7
8     if (strcmp(args, "r") == 0) {
9         // 打印所有寄存器
10        for (int i = 0; i < 8; i++) {
11            printf("%s 0x%x\n", regsl[i], reg_l(i));
12        }
13        printf("eip 0x%x\n", cpu.eip);
14        for (int i = 0; i < 8; i++) {
15            printf("%s 0x%x\n", regsw[i], reg_w(i));
16        }
17        for (int i = 0; i < 8; i++) {
18            printf("%s 0x%x\n", regsb[i], reg_b(i));
19        }
20    } else if (strcmp(args, "w") == 0) {
21        print_wp();
22    } else {
23        printf("Unknown argument '%s'. Usage: info r or info w\n", args);
24    }
25 }
```

```

24     }
25
26     return 0;
27 }
28

```

cmd_info 函数用于显示模拟器当前的重要运行状态，主要包括寄存器信息和监视点信息。函数首先调用 trim 清理参数；如果参数为空或者长度为零，则直接打印帮助提示。然后，当参数为 r 时，函数遍历所有通用寄存器，并分别通过 regsl、regsw、regsb 这些名称数组以及 reg_l、reg_w、reg_b 这类访问宏，输出寄存器的不同位宽视图，同时还打印 eip，用于观察当前指令执行位置。当参数为 w 时，函数则调用 print_wp() 输出所有监视点和断点信息，以使用户检查当前已经设置的调试条件。如果参数不是 r 也不是 w，函数会给出错误提示，防止用户输入非法参数却得不到反馈。

运行结果如下：

```

./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 01:11:40, Apr 7 2026
For help, type "help"
(nemu) info r
eax 0x5d6bb9b4
ecx 0x2e62125f
edx 0x25021e62
ebx 0x7862405f
esp 0x7ec8694a
ebp 0x4f2c4c7f
esi 0x3e4e76ce
edi 0x370d1666
eip 0x100000
ax 0xb9b4
cx 0x125f
dx 0x1e62
bx 0x405f
sp 0x694a
bp 0x4c7f
si 0x76ce
di 0x1666
al 0xb4
cl 0x5f
dl 0x62
bl 0x5f
ah 0xb9
ch 0x12
dh 0x1e
bh 0x40
(nemu)

```

2.2.3 扫描内存 x

Listing 6: cmd_x

```

1 static int cmd_x(char *args) {
2     if (args == NULL) {
3         printf("Usage: x N EXPR (N: number of bytes, EXPR: starting address)\n");
4         return 0;
5     }
6
7     char *num_str = strtok(args, " ");
8     char *addr_str = strtok(NULL, " ");
9
10    if (num_str == NULL || addr_str == NULL) {
11        printf("Usage: x N EXPR (N: number of bytes, EXPR: starting address)\n");

```

```
12     return 0;
13 }
14
15 // N 转整数
16 int num = atoi(num_str);
17 if (num <= 0) {
18     printf("Invalid number of bytes: %s\n", num_str);
19     return 0;
20 }
21
22 // EXPR 转十六进制
23 uint32_t addr;
24 if (sscanf(addr_str, "%x", &addr) != 1) {
25     printf("Invalid address: %s\n", addr_str);
26     return 0;
27 }
28
29 for (int i = 0; i < num; i++) {
30     uint8_t data = vaddr_read(addr, 1); // 每次读取 1 字节
31     printf("0x%02x ", data);
32     addr += 1;
33
34     // 每 16 字节换行
35     if ((i + 1) % 16 == 0) {
36         printf("\n");
37     }
38 }
39
40 if (num % 16 != 0) {
41     printf("\n");
42 }
43
44 return 0;
45 }
46
```

cmd_x 函数实现了内存扫描功能，用于按地址连续读取内存内容。函数首先检查参数是否为空，如果没有提供参数就直接提示正确用法。接着，它使用 strtok 将参数拆分为两个部分：第一个部分表示要读取的字节数 N，第二个部分表示起始地址表达式 EXPR。随后，函数把字节数字符串通过 atoi 转换成整数，并检查其是否大于 0。起始地址则通过 sscanf(addr_str, "%x", &addr) 按十六进制格式解析为无符号地址。完成参数转换后，函数进入循环，每次调用 vaddr_read(addr, 1) 读取一个字节的数，并以 0x%02x 的格式打印出来，同时将地址递增 1，继续读取下一字节。为了让输出更加规整，它每输出 16 个字节就自动换行一次。

运行结果如下：


```

./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 03:39:55, Apr 21 2026
For help, type "help"
(nemu) x 10 0x100000
0xb8 0x34 0x12 0x00 0x00 0xb9 0x27 0x00 0x10 0x00
(nemu) x 39 0x100000
0xb8 0x34 0x12 0x00 0x00 0xb9 0x27 0x00 0x10 0x00 0x89 0x01 0xc6 0xc7 0x41 0x04
0x01 0x00 0xbb 0x02 0x00 0x00 0x00 0xc6 0xc7 0x84 0x99 0x00 0xe0 0xff 0xff 0x01
0x00 0xb8 0x00 0x00 0x00 0xd6
(nemu) q

```

对照 nemu/src/monitor/monitor.c 里的默认镜像：

```

static inline int load_default_img() {
    const uint8_t img[] = {
        0xb8, 0x34, 0x12, 0x00, 0x00,           // 100000: movl $0x1234,%eax
        0xb9, 0x27, 0x00, 0x10, 0x00,           // 100005: movl $0x100027,%ecx
        0x89, 0x01,                             // 10000a: movl %eax, (%ecx)
        0xc6, 0xc7, 0x41, 0x04, 0x01, 0x00,     // 10000c: movw $0x1,0x4(%ecx)
        0xbb, 0x02, 0x00, 0x00, 0x00,           // 100012: movl $0x2,%ebx
        0xc6, 0xc7, 0x84, 0x99, 0x00, 0xe0,     // 100017: movw $0x1,-0x2000(%ecx,%ebx,4)
        0xff, 0xff, 0x01, 0x00,
        0xb8, 0x00, 0x00, 0x00, 0x00,           // 100021: movl $0x0,%eax
        0xd6,                                   // 100026: nemu_trap
    };
}

```

可见成功实现。

3 PA1-2

3.1 表达式求值

首先在 ui.c 文件中添加表达式求值的命令：

Listing 7: cmd_p

```

1 static int cmd_p(char *args) {
2     args = trim(args);
3     if (args == NULL || args[0] == '\0') {
4         printf("Usage: p EXPR\n");
5         return 0;
6     }
7     bool finished = true;
8     uint32_t ans = expr(args, &finished);
9     if (!finished) {
10        printf("please check your expression..!\n");
11    } else {
12        printf("0x%x (%u)\n", ans, ans);
13    }
14    return 0;
15 }
16

```

作用是接收用户在调试器命令行中输入的表达式，完成基本合法性检查后，将表达式交给底层求值模块 `expr()` 处理。

随后需要在 `nemu/src/monitor/debug/expr.c` 中完善 `token` 类型定义：

Listing 8: token 类型定义

```

1  enum {
2      TK_NOTYPE = 256, TK_EQ,
3
4      /* TODO: Add more token types */
5      TK_NEQ, TK_AND, TK_OR, TK_NOT,
6      TK_DECIMAL,      // 十进制整数
7      TK_HEX,           // 十六进制整数
8      TK_REG,           // 寄存器
9      TK_LPAREN,        // (
10     TK_RPAREN,         // )
11     TK_NEG,            // 负号
12     TK_DEREF           // 指针解引用
13 };
14

```

expr.c 中的 rules[] 数组定义了字符串到 token 的映射规则，是词法分析阶段的核心配置。该数组使用正则表达式描述每一种合法 token 的形式，也要完善好：

Listing 9:]rules[]

```

1  static struct rule {
2      char *regex;
3      int token_type;
4  } rules[] = {
5
6      /* TODO: Add more rules.
7       * Pay attention to the precedence level of different rules.
8       */
9
10     {" +", TK_NOTYPE},      // 空格
11     {"0[xX][0-9a-fA-F]+"}, // 十六进制整数
12     {"\\$[a-zA-Z]+"},      // 寄存器
13     {"[0-9]+"},             // 十进制整数
14     {"=="}, TK_EQ},         // 等于
15     {"!="}, TK_NEQ},        // 不等于
16     {"&&"}, TK_AND},        // 逻辑与
17     {"\\|\\|\\|"}, TK_OR},  // 逻辑或
18     {"!"}, TK_NOT},         // 逻辑非
19     {"\\+"}, '+'},          // +
20     {"-"}, '-'},            // -
21     {"\\*"}, '*'},          // *
22     {"\\/"}, '/'},          // /
23     {"\\("}, TK_LPAREN},    // 左括号
24     {"\\)"}, TK_RPAREN},    // 右括号
25 };
26

```

词法分析完善好就可以开始完善实现表达式求值功能，该功能是从简单表达式求值-考虑负号-考

考虑十六进制数-考虑一元运算符等等最终实现复杂表达式求值的，这里直接展示最终阶段：

Listing 10: expr.c 中的 make_token

```

1 static bool make_token(char *e) {
2     int position = 0;
3     int i;
4     regmatch_t pmatch;
5
6     nr_token = 0;
7
8     while (e[position] != '\0') {
9         /* Try all rules one by one. */
10        for (i = 0; i < NR_REGEX; i++) {
11            if (regexec(&re[i], e + position, 1, &pmatch, 0) == 0 && pmatch.rm_so == 0) {
12                char *substr_start = e + position;
13                int substr_len = pmatch.rm_eo;
14
15                Log("match rules[%d] = \"%s\" at position %d with len %d: %.s",
16                    i, rules[i].regex, position, substr_len, substr_start);
17                position += substr_len;
18
19                /* TODO: Now a new token is recognized with rules[i]. Add codes
20                 * to record the token in the array `tokens'. For certain types
21                 * of tokens, some extra actions should be performed.
22                 */
23
24                // 如果匹配到的是空格，直接跳过
25                if (rules[i].token_type == TK_NOTYPE) {
26                    break;
27                }
28
29                // 记录 token 信息
30                tokens[nr_token].type = rules[i].token_type;
31
32                // 如果是整数|寄存器，记录其值
33                if (rules[i].token_type == TK_DECIMAL || rules[i].token_type == TK_HEX ||
34                    rules[i].token_type == TK_REG) {
35                    if (substr_len >= sizeof(tokens[nr_token].str)) {
36                        printf("Error: token too long at position %d\n", position);
37                        return false; // 防止缓冲区溢出
38                    }
39                    strncpy(tokens[nr_token].str, substr_start, substr_len);
40                    tokens[nr_token].str[substr_len] = '\0';
41                }
42
43                // 判断负号
44                if (rules[i].token_type == '-') {
45                    if (nr_token == 0 || tokens[nr_token - 1].type == '+' || tokens[nr_token - 1].type
46                        == '-' || tokens[nr_token - 1].type == '*' || tokens[nr_token - 1].type == '/' ||

```

```

tokens[nr_token - 1].type == TK_LPAREN || tokens[nr_token - 1].type == TK_EQ ||
tokens[nr_token - 1].type == TK_NEQ || tokens[nr_token - 1].type == TK_AND || tokens[nr_token
- 1].type == TK_OR) {
45     tokens[nr_token].type = TK_NEG; // 当前是负号
46 }
47 }
48
49 // 检测指针解引用
50 if (rules[i].token_type == '*') {
51     if (nr_token == 0 || tokens[nr_token - 1].type == '+' || tokens[nr_token - 1].type
== '-' || tokens[nr_token - 1].type == '*' || tokens[nr_token - 1].type == '/' ||
tokens[nr_token - 1].type == TK_LPAREN || tokens[nr_token - 1].type == TK_EQ ||
tokens[nr_token - 1].type == TK_NEQ || tokens[nr_token - 1].type == TK_AND || tokens[nr_token
- 1].type == TK_OR) {
52         tokens[nr_token].type = TK_DEREF; // 当前是指针解引用
53     }
54 }
55
56 nr_token++;
57 if (nr_token >= sizeof(tokens) / sizeof(tokens[0])) {
58     printf("Error: too many tokens\n");
59     return false; // 防止 tokens 数组溢出
60 }
61
62 break;
63 }
64 }
65
66 if (i == NR_REGEX) {
67     printf("no match at position %d\n%s\n%*.s^\n", position, e, position, "");
68     return false;
69 }
70 }
71
72 return true;
73 }
74

```

make_token 函数负责把输入表达式字符串扫描并分解为一个个 token，是整个表达式求值流程中的第一步。函数从字符串开头开始逐字符扫描，每次都尝试用 rules[] 中定义的正则规则依次匹配当前位置的内容；一旦某条规则匹配成功，就记录匹配长度并将扫描位置向前推进。对于匹配到空格的情况，函数直接跳过，不生成 token。对于其余 token，函数会把 token 类型写入 tokens[nr_token].type，并在遇到数字或寄存器这类需要保留原始文本的 token 时，把对应子串复制到 tokens[nr_token].str 中，以便后续进行数值转换或寄存器名匹配。

这段代码中最关键的处理是对一元运算符的识别：当 - 出现在表达式开头，或者前一个 token 是运算符、左括号、逻辑符号等情况时，当前 - 不再被视为减法，而是被重标记为 TK_NEG，表示一元负号；同理，当 * 出现在表达式开头，或者前一个 token 处于类似“运算符后接操作数”的环境时，它会被重

标记为 TK_DEREF，表示指针解引用。这样的上下文判定是表达式解析中非常重要的一步，解决符号多义性问题，使解释器能够正确区分 $a - b$ 和 $-b$ ，以及 $a * b$ 和 $*addr$ 。如果某一位置无法匹配任何规则，函数会输出错误位置并返回失败，说明表达式中存在非法字符或语法片段。总体上，make_token 的工作就是完成“词法切分”，把原始字符串转化为后续递归分析可直接使用的 token 序列。

Listing 11: check_parentheses

```

1  static bool check_parentheses(int p, int q) {
2      if (tokens[p].type != TK_LPAREN || tokens[q].type != TK_RPAREN) {
3          return false; // 不以括号包围
4      }
5
6      int balance = 0;
7      for (int i = p; i <= q; i++) {
8          if (tokens[i].type == TK_LPAREN) {
9              balance++;
10         } else if (tokens[i].type == TK_RPAREN) {
11             balance--;
12         }
13
14         if (balance < 0) {
15             return false;
16         }
17
18         if (balance == 0 && i < q) {
19             return false;
20         }
21     }
22
23     return balance == 0;
24 }
25

```

check_parentheses 函数用于判断一个 token 区间是否被一对完整且合法的外层括号包围。函数首先检查区间首尾 token 是否分别为左括号和右括号，如果不是，则说明整个区间不可能被一对外层括号完整包住，直接返回 false。随后，函数通过一个 balance 变量记录括号嵌套深度：遇到左括号时加一，遇到右括号时减一，并在扫描过程中判断括号是否出现提前闭合或嵌套不平衡。如果在扫描中 balance 变成负数，说明出现了多余的右括号，表达式非法；如果在 balance 重新变为 0 时还没有扫描到区间末尾，就说明最外层括号并没有真正包住整个子表达式，而只是包住了其中一部分，因此也不能简单地视为整体括号表达式。只有当整个区间扫描结束后，balance 恰好回到 0，函数才返回 true，表示该区间可以视为一个被外层括号完全包裹的表达式。

Listing 12: find_dominant_operator

```

1  static int find_dominant_operator(int p, int q) {
2      int op = -1;
3      int min_priority = 100; // 假设优先级最大值为 100
4      int balance = 0;
5

```

```

6   for (int i = p; i <= q; i++) {
7       if (tokens[i].type == TK_LPAREN) balance++;
8       if (tokens[i].type == TK_RPAREN) balance--;
9       if (balance > 0) continue; // 括号内的运算符跳过
10
11      int priority = -1;
12      switch (tokens[i].type) {
13          case TK_OR: priority = 0; break; // ||
14          case TK_AND: priority = 1; break; // &&
15          case TK_EQ: case TK_NEQ: priority = 2; break; // ==, !=
16          case '+': case '-': priority = 3; break; // +, -
17          case '*': case '/': priority = 4; break; // *, /
18          case TK_NEG: case TK_NOT: case TK_DEREF: priority = 5; break; // 单目运算符
19      }
20
21      if (priority != -1 && priority <= min_priority) {
22          min_priority = priority;
23          op = i;
24      }
25  }
26
27  return op;
28  }
29

```

find_dominant_operator 函数用于在一个表达式子区间中寻找主运算符，也就是递归分治求值时切分左右子表达式的关键位置。函数扫描 [p, q] 范围内的所有 token，同时维护括号深度 balance，一旦进入括号内部就跳过其中的运算符，只考虑最外层的运算符，这样可以避免子表达式内部的运算优先级干扰整体切分。对于每个候选 token，函数根据其类型赋予不同的优先级数值：逻辑或最低，其次是逻辑与，再是相等/不等比较，然后是加减，再是乘除，而一元负号、逻辑非和解引用的优先级最高。函数使用“优先级越低越优先作为主运算符”，并在扫描过程中不断更新当前找到的最低优先级运算符位置。

Listing 13: eval

```

1   static uint32_t eval(int p, int q, bool *success) {
2       if (p > q) {
3           *success = false;
4           return 0; // 非法表达式
5       }
6
7       if (p == q) {
8           if (tokens[p].type == TK_DECIMAL) {
9               *success = true;
10              return atoi(tokens[p].str); // 转换为整数
11          }
12          if (tokens[p].type == TK_HEX) {
13              *success = true;
14              uint32_t val;

```

```

15     sscanf(tokens[p].str, "%x", &val);
16     return val;
17 }
18 if (tokens[p].type == TK_REG) {
19     *success = true;
20     if (strcmp(tokens[p].str, "$eax") == 0) return cpu.eax;
21     if (strcmp(tokens[p].str, "$ecx") == 0) return cpu.ecx;
22     if (strcmp(tokens[p].str, "$edx") == 0) return cpu.edx;
23     if (strcmp(tokens[p].str, "$ebx") == 0) return cpu.ebx;
24     if (strcmp(tokens[p].str, "$esp") == 0) return cpu.esp;
25     if (strcmp(tokens[p].str, "$ebp") == 0) return cpu.ebp;
26     if (strcmp(tokens[p].str, "$esi") == 0) return cpu.esi;
27     if (strcmp(tokens[p].str, "$edi") == 0) return cpu.edi;
28     if (strcmp(tokens[p].str, "$eip") == 0) return cpu.eip;
29     *success = false;
30     return 0;
31 }
32 *success = false;
33 return 0;
34 }
35
36 if (check_parentheses(p, q)) {
37     // 去掉外层括号
38     return eval(p + 1, q - 1, success);
39 }
40
41 int op = find_dominant_operator(p, q);
42 if (op == -1) {
43     *success = false;
44     return 0; // 非法表达式
45 }
46
47 if (tokens[op].type == TK_NEG) {
48     // 处理负号
49     uint32_t val = eval(op + 1, q, success);
50     if (!*success) return 0;
51     return -val;
52 }
53
54 if (tokens[op].type == TK_NOT) {
55     uint32_t val = eval(op + 1, q, success);
56     if (!*success) return 0;
57     return !val;
58 }
59
60 if (tokens[op].type == TK_DEREF) {
61     uint32_t addr = eval(op + 1, q, success);
62     if (!*success) return 0;
63     return vaddr_read(addr, 4); // 从内存中读取 4 字节

```

```

64     }
65
66     uint32_t val1 = eval(p, op - 1, success);
67     if (!*success) return 0;
68
69     uint32_t val2 = eval(op + 1, q, success);
70     if (!*success) return 0;
71
72     switch (tokens[op].type) {
73         case TK_EQ: return val1 == val2;
74         case TK_NEQ: return val1 != val2;
75         case TK_AND: return val1 && val2;
76         case TK_OR: return val1 || val2;
77         case '+': return val1 + val2;
78         case '-': return val1 - val2;
79         case '*': return val1 * val2;
80         case '/':
81             if (val2 == 0) {
82                 *success = false;
83                 return 0;
84             }
85             return val1 / val2;
86         default:
87             *success = false;
88             return 0;
89     }
90 }
91

```

eval 函数是整个表达式求值模块的核心递归函数，它负责根据 token 区间 [p, q] 的语法结构逐层计算表达式值。函数首先处理非法区间情况，如果 $p > q$ ，说明表达式片段为空或切分错误，直接判定失败；当 $p == q$ 时，表示区间只剩下一个 token，此时只需要根据 token 类型直接返回对应的值即可。若 token 是十进制数，就通过 atoi 转换为整数；若是十六进制数，就用 sscanf 将字符串解析为数值；若是寄存器名，则通过一系列字符串比较把 \$eax、\$ecx、\$edx、\$ebx、\$esp、\$ebp、\$esi、\$edi 和 \$eip 映射为 CPU 中对应寄存器的当前值。若 token 类型不合法，则返回失败。当当前区间不止一个 token 时，函数先调用 check_parentheses 判断是否可以直接去掉外层括号；如果可以，就递归求值括号内部内容。否则，函数调用 find_dominant_operator 寻找当前区间的主运算符，并根据该运算符类型进行分类处理。若主运算符是 TK_NEG，则说明当前表达式是一个一元负号表达式，函数只需对右侧子表达式求值后取负即可；若是 TK_NOT，则对右侧结果取逻辑非；若是 TK_DEREF，则先求出地址值，再调用 vaddr_read(addr, 4) 读取该地址处的 4 字节内容，实现对内存的间接访问。对于二元运算符，函数会分别递归计算左右子表达式，再根据运算符类型执行相应操作，包括等于、不等于、逻辑与、逻辑或、加、减、乘和除。除法运算中还专门检查除数是否为 0，以避免非法除零错误。

整个 eval 的递归逻辑本质上是在构建一棵隐式表达式树：每次找到主运算符后，左子树和右子树分别递归求值，再将结果合并。由于该函数还支持寄存器和内存访问，因此它不仅能计算纯算术表达式，还能直接分析运行时状态，例如 \$eip + 4、*\$esp 之类的调试表达式。

Listing 14: expr

```

1  uint32_t expr(char *e, bool *success) {
2      if (!make_token(e)) {
3          *success = false;
4          return 0;
5      }
6
7      /* TODO: Insert codes to evaluate the expression. */
8      return eval(0, nr_token - 1, success); // 从整个表达式开始求值
9
10     return 0;
11 }
12

```

expr 函数是表达式求值模块对外的统一入口。函数首先调用 make_token(e) 将输入字符串切分成 token 序列, 如果词法分析失败, 说明输入表达式中存在非法字符、未匹配符号或 token 数量超限等问题, 此时直接将 success 置为 false 并返回 0。若词法分析成功, 则调用 eval(0, nr_token - 1, success) 对整个 token 区间进行递归求值, 也就是从第一个 token 到最后一个 token 完整计算表达式的值。

对实现的表达式求值功能进行几例测试, 结果如下:

```

(nemu) p (2+3)*(4+5*(6-2))
[src/monitor/debug/expr.c,94,make_token] match rules[13] = "(" at position 0 with len 1: (
[src/monitor/debug/expr.c,94,make_token] match rules[3] = "[0-9]+" at position 1 with len 1: 2
[src/monitor/debug/expr.c,94,make_token] match rules[9] = "+" at position 2 with len 1: +
[src/monitor/debug/expr.c,94,make_token] match rules[3] = "[0-9]+" at position 3 with len 1: 3
[src/monitor/debug/expr.c,94,make_token] match rules[14] = ")" at position 4 with len 1: )
[src/monitor/debug/expr.c,94,make_token] match rules[11] = "*" at position 5 with len 1: *
[src/monitor/debug/expr.c,94,make_token] match rules[13] = "(" at position 6 with len 1: (
[src/monitor/debug/expr.c,94,make_token] match rules[3] = "[0-9]+" at position 7 with len 1: 4
[src/monitor/debug/expr.c,94,make_token] match rules[9] = "+" at position 8 with len 1: +
[src/monitor/debug/expr.c,94,make_token] match rules[3] = "[0-9]+" at position 9 with len 1: 5
[src/monitor/debug/expr.c,94,make_token] match rules[11] = "*" at position 10 with len 1: *
[src/monitor/debug/expr.c,94,make_token] match rules[13] = "(" at position 11 with len 1: (
[src/monitor/debug/expr.c,94,make_token] match rules[3] = "[0-9]+" at position 12 with len 1: 6
[src/monitor/debug/expr.c,94,make_token] match rules[10] = "-" at position 13 with len 1: -
[src/monitor/debug/expr.c,94,make_token] match rules[3] = "[0-9]+" at position 14 with len 1: 2
[src/monitor/debug/expr.c,94,make_token] match rules[14] = ")" at position 15 with len 1: )
[src/monitor/debug/expr.c,94,make_token] match rules[14] = ")" at position 16 with len 1: )
120
(nemu) p -(-1)
[src/monitor/debug/expr.c,94,make_token] match rules[10] = "-" at position 0 with len 1: -
[src/monitor/debug/expr.c,94,make_token] match rules[13] = "(" at position 1 with len 1: (
[src/monitor/debug/expr.c,94,make_token] match rules[10] = "-" at position 2 with len 1: -
[src/monitor/debug/expr.c,94,make_token] match rules[3] = "[0-9]+" at position 3 with len 1: 1
[src/monitor/debug/expr.c,94,make_token] match rules[14] = ")" at position 4 with len 1: )
1
(nemu) p 1&&0
[src/monitor/debug/expr.c,94,make_token] match rules[3] = "[0-9]+" at position 0 with len 1: 1
[src/monitor/debug/expr.c,94,make_token] match rules[6] = "&&" at position 1 with len 2: &&
[src/monitor/debug/expr.c,94,make_token] match rules[3] = "[0-9]+" at position 3 with len 1: 0
0
(nemu) p 1+2==4||3==3
[src/monitor/debug/expr.c,94,make_token] match rules[3] = "[0-9]+" at position 0 with len 1: 1
[src/monitor/debug/expr.c,94,make_token] match rules[9] = "+" at position 1 with len 1: +
[src/monitor/debug/expr.c,94,make_token] match rules[3] = "[0-9]+" at position 2 with len 1: 2
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 3 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[3] = "[0-9]+" at position 5 with len 1: 4
[src/monitor/debug/expr.c,94,make_token] match rules[7] = "||" at position 6 with len 2: ||
[src/monitor/debug/expr.c,94,make_token] match rules[3] = "[0-9]+" at position 8 with len 1: 3
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 9 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[3] = "[0-9]+" at position 11 with len 1: 3
1

```

4 PA1-3

4.1 监视点和断点

要实现监视点和断点的功能，首先还是要在 ui.c 中补充 cmd_w、cmd_b、cmd_d 并添加在命令表里：

Listing 15: 补充 ui.c

```

1  static int cmd_w(char *args) {
2      args = trim(args);
3      WP *wp = set_watchpoint(args);
4      if (wp != NULL) {
5          printf("Watchpoint %d: %s = 0x%08x\n", wp->NO, wp->expr, wp->last_value);
6      }
7      return 0;
8  }
9
10 static int cmd_b(char *args) {
11     args = trim(args);
12     if (args == NULL || args[0] == '\0') {
13         printf("Usage: b EXPR\n");
14         return 0;
15     }
16
17     if (args[0] == '*') {
18         args = trim(args + 1);
19     }
20
21     bool success = true;
22     vaddr_t addr = expr(args, &success);
23     if (!success) {
24         printf("Bad breakpoint address: %s\n", args);
25         return 0;
26     }
27
28     WP *wp = set_breakpoint(addr);
29     if (wp != NULL) {
30         printf("Breakpoint %d at 0x%08x\n", wp->NO, wp->addr);
31     }
32     return 0;
33 }
34
35 static int cmd_d(char *args) {
36     args = trim(args);
37     if (args == NULL || args[0] == '\0') {
38         printf("Usage: d N\n");
39         return 0;
40     }
41 }

```

```

42     char *end = NULL;
43     int no = strtol(args, &end, 10);
44     end = trim(end);
45     if (no < 0 || end == NULL || end[0] != '\0') {
46         printf("Invalid watchpoint/breakpoint number: %s\n", args);
47         return 0;
48     }
49
50     if (delete_wp(no)) {
51         printf("Deleted watchpoint/breakpoint %d\n", no);
52     }
53     else {
54         printf("No watchpoint/breakpoint number %d\n", no);
55     }
56     return 0;
57 }
58

```

cmd_w 函数实现了监视点命令的用户入口，它接收用户输入的表达式并将其注册为一个新的监视点，从而在程序运行过程中持续观察该表达式的值是否发生变化。函数首先调用 trim 去除参数首尾空白字符；随后直接调用 set_watchpoint(args)，由监视点管理模块完成表达式合法性检查、初始求值以及对象分配。如果监视点创建成功，函数会打印监视点编号、表达式内容以及当前值。

cmd_b 函数用于设置断点，它的实现方式与监视点类似。函数首先对参数进行 trim，然后检查用户是否提供了有效输入，如果输入为空，则打印使用说明。接下来，函数还支持一种更贴近调试器习惯的写法，即用户可以使用 b *addr 的形式指定地址断点，因此当参数以 * 开头时，函数会把这个符号去掉后继续处理。随后，程序调用 expr(args, &success) 对输入表达式求值，并把结果解释为断点地址；如果表达式求值失败，则说明输入并不是一个合法地址，函数会给出错误提示并终止操作。如果地址解析成功，就调用 set_breakpoint(addr) 创建一个断点对象，并在成功后输出断点编号和断点地址。

cmd_d 函数用于删除已存在的监视点或断点，使调试器中的调试对象可以按编号进行回收。函数首先调用 trim 清理输入参数，并检查是否为空。随后，函数使用 strtol 将字符串转换为整数编号，并通过 end 指针确认输入中是否存在非法字符，保证删除命令的参数是严格合法的数字。若编号小于 0，或者编号后仍然残留其他非空字符，函数就会认为输入非法并输出错误提示。参数合法后，函数调用 delete_wp(no) 在监视点链表中查找对应编号的对象，如果查找并删除成功，就输出删除成功的信息；否则则说明当前并不存在该编号的监视点或断点。

接下来是对 watchpoint.c 中的核心功能的实现：

Listing 16: init_wp_pool

```

1     void init_wp_pool() {
2         int i;
3         for (i = 0; i < NR_WP; i++) {
4             wp_pool[i].NO = i;
5             wp_pool[i].next = &wp_pool[i + 1];
6         }
7         wp_pool[NR_WP - 1].next = NULL;
8
9         head = NULL;

```

```

10     free_ = wp_pool;
11     skip_current_bp = false;
12     skip_bp_addr = 0;
13 }
14

```

init_wp_pool 函数负责初始化监视点对象池。它遍历静态数组 wp_pool，将其中每一个元素按照顺序链接成一个空闲链表，同时为每个监视点对象分配固定编号 NO，使后续创建的监视点能够直接通过编号识别和管理。最后一个节点的 next 被置为 NULL，表示空闲链表结束。初始化结束后，head 被设为 NULL，表示当前已经使用的监视点链表为空，而 free_ 指向 wp_pool 的首元素，表示所有监视点对象都处于可分配状态。除此之外，函数还把 skip_current_bp 和 skip_bp_addr 初始化为默认值，这两个变量用于断点命中后的跳过控制，防止断点在某些情况下重复触发。

Listing 17: new_wp

```

1  WP *new_wp() {
2      if (free_ == NULL) {
3          printf("No free watchpoint.\n");
4          return NULL;
5      }
6
7      WP *wp = free_;
8      free_ = free_->next;
9
10     wp->next = head;
11     head = wp;
12
13     wp->type = WP_TYPE_WATCHPOINT;
14     wp->expr[0] = '\0';
15     wp->last_value = 0;
16     wp->addr = 0;
17     return wp;
18 }
19

```

new_wp 函数用于从空闲对象池中申请一个新的监视点或断点节点。函数首先检查 free_ 是否为空，如果空闲链表已经没有可用节点，则打印提示信息并返回 NULL，说明当前无法再创建新的监视对象。若空闲链表中仍有节点可用，函数就从 free_ 所指向的节点中取出一个对象，并把 free_ 前移到下一个空闲节点，实现空闲链表的弹出操作。随后，该节点被插入到已使用链表的头部，也就是让它成为 head 指向的新节点。接下来，函数会对该节点进行初始化：把类型设为监视点，把表达式字符串清空，把上一次记录的值设为 0，把地址字段设为 0。这样，返回给上层的对象就已经是一个结构完整、可直接使用的调试对象。

Listing 18: free_wp

```

1  void free_wp(WP *wp) {
2      if (wp == NULL) {
3          return;
4      }

```

```

5
6     WP **cur = &head;
7     while (*cur != NULL && *cur != wp) {
8         cur = &(*cur)->next;
9     }
10
11     if (*cur == NULL) {
12         return;
13     }
14
15     *cur = wp->next;
16     wp->next = free_;
17     free_ = wp;
18 }
19

```

free_wp 函数负责回收一个已经使用的监视点或断点节点，并将其重新归还到空闲对象池中。函数首先判断传入指针是否为空，如果为空则直接返回，避免无效操作。接着，它通过指针遍历已使用链表 head，寻找目标节点所在的位置，直到找到与 wp 相同的节点为止。如果遍历结束后仍未找到该节点，说明传入对象并不在当前活动链表中，函数也会直接返回。找到目标节点后，函数通过修改前驱节点的 next 指针，把目标节点从已使用链表中摘除，然后再把它插回到空闲链表 free_ 的头部，实现对象的回收和复用。

Listing 19: set_watchpoint

```

1     WP *set_watchpoint(char *expr_str) {
2         if (expr_str == NULL || expr_str[0] == '\0') {
3             printf("Usage: w EXPR\n");
4             return NULL;
5         }
6
7         if (strlen(expr_str) >= sizeof(wp_pool[0].expr)) {
8             printf("Expression is too long.\n");
9             return NULL;
10        }
11
12        bool success = true;
13        uint32_t value = expr(expr_str, &success);
14        if (!success) {
15            printf("Bad expression: %s\n", expr_str);
16            return NULL;
17        }
18
19        WP *wp = new_wp();
20        if (wp == NULL) {
21            return NULL;
22        }
23
24        wp->type = WP_TYPE_WATCHPOINT;

```

```

25     strcpy(wp->expr, expr_str);
26     wp->last_value = value;
27     wp->addr = 0;
28     return wp;
29 }
30

```

set_watchpoint 函数负责根据用户提供的表达式创建一个新的监视点，它把表达式合法性检查、初始求值和对象分配整合在一起，构成完整的监视点创建流程。函数首先检查输入是否为空，如果用户没有提供表达式，则打印用法提示并返回 NULL。随后，它检查表达式长度是否超出监视点对象中字符串缓冲区的容量，防止表达式过长引起数组越界。接着，函数调用 expr(expr_str, &success) 对表达式进行求值，如果表达式本身有语法错误、非法 token 或其他求值失败情况，就直接拒绝创建监视点并输出错误提示。只有当表达式能够成功求值时，函数才调用 new_wp() 从对象池中分配一个新节点。若对象池已经耗尽，函数同样返回失败。

当节点成功获取后，函数将该节点标记为监视点类型，并将原始表达式字符串复制进去，同时把表达式当前的计算结果保存到 last_value 字段中，这样后续执行过程中就可以通过比较新旧值判断是否触发。addr 字段在监视点场景下并不使用，因此被置为 0。

Listing 20: set_breakpoint

```

1  WP *set_breakpoint(vaddr_t addr) {
2      WP *wp = new_wp();
3      if (wp == NULL) {
4          return NULL;
5      }
6
7      wp->type = WP_TYPE_BREAKPOINT;
8      snprintf(wp->expr, sizeof(wp->expr), "$eip == 0x%08x", addr);
9      wp->last_value = 0;
10     wp->addr = addr;
11     return wp;
12 }
13

```

set_breakpoint 函数用于创建断点对象，它和 set_watchpoint 在结构上相似。函数首先调用 new_wp() 从对象池中申请一个空闲节点，如果没有可用节点，则直接返回失败。与监视点不同，断点并不保存任意复杂表达式，而是以当前指令地址 eip 是否等于目标地址为判定条件，因此该函数会直接把断点对象的类型设置为 WP_TYPE_BREAKPOINT。随后，它使用 snprintf 把断点条件格式化为字符串 \$eip == 0x%08x 并保存到 expr 字段中，这样断点在表现形式上也被统一为一个表达式，便于打印和管理。last_value 字段在断点场景下通常没有直接意义，因此被置为 0，而 addr 字段则保存实际断点地址，供后续检查命中时直接比较。

Listing 21: delete_wp

```

1  bool delete_wp(int no) {
2      WP *cur = head;
3      while (cur != NULL) {
4          if (cur->NO == no) {

```

```

5     free_wp(cur);
6     return true;
7 }
8     cur = cur->next;
9 }
10
11     return false;
12 }
13

```

delete_wp 函数负责按编号删除一个监视点或断点。函数从活动链表的头节点 head 开始遍历，逐个比较节点的 NO 字段是否等于用户输入的编号。如果找到匹配节点，就调用 free_wp(cur) 将该节点从活动链表中移除，并回收到空闲链表中，然后返回 true 表示删除成功。若遍历结束后仍未找到对应编号的节点，则返回 false，说明当前不存在该对象。

Listing 22: print_wp

```

1 void print_wp(void) {
2     if (head == NULL) {
3         printf("No watchpoints or breakpoints.\n");
4         return;
5     }
6
7     printf("%-4s %-12s %-12s %s\n", "NO", "TYPE", "VALUE", "EXPR");
8     for (WP *cur = head; cur != NULL; cur = cur->next) {
9         if (cur->type == WP_TYPE_BREAKPOINT) {
10             printf("%-4d %-12s 0x%08x   %s\n",
11                 cur->NO, "breakpoint", cur->addr, cur->expr);
12         }
13         else {
14             printf("%-4d %-12s 0x%08x   %s\n",
15                 cur->NO, "watchpoint", cur->last_value, cur->expr);
16         }
17     }
18 }
19

```

print_wp 函数用于输出当前所有监视点和断点的信息。函数首先检查活动链表 head 是否为空，如果没有任何监视对象，则直接提示当前不存在监视点或断点。若链表中有对象，则先打印表头，包括编号、类型、当前值和表达式等字段，使输出格式更加规整。随后，它遍历所有活动节点，并根据节点类型分别打印不同内容：如果是断点，则输出断点编号、类型字符串“breakpoint”、断点地址以及对应表达式；如果是监视点，则输出编号、类型字符串“watchpoint”、当前保存的值以及表达式本身。

对于监视点来说，last_value 表示上一次记录的表达式结果；对于断点来说，addr 表示命中的目标地址。通过统一的打印接口，用户能够在调试过程中快速查看所有活跃调试对象，从而知道程序为何在某个位置停下，或者为什么某个表达式变化会触发中断。print_wp 因此是 info w 命令的底层支撑，也是调试器可视化能力的重要组成部分。

Listing 23: check_wp


```

1  bool check_wp(void) {
2      bool stop = false;
3
4      for (WP *cur = head; cur != NULL; cur = cur->next) {
5          if (cur->type != WP_TYPE_WATCHPOINT) {
6              continue;
7          }
8
9          bool success = true;
10         uint32_t value = expr(cur->expr, &success);
11         if (!success) {
12             printf("Watchpoint %d expression error: %s\n", cur->NO, cur->expr);
13             stop = true;
14             continue;
15         }
16
17         if (value != cur->last_value) {
18             printf("Watchpoint %d triggered: %s\n", cur->NO, cur->expr);
19             printf("Old value = 0x%08x\n", cur->last_value);
20             printf("New value = 0x%08x\n", value);
21             cur->last_value = value;
22             stop = true;
23         }
24     }
25
26     return stop;
27 }
28

```

check_wp 函数负责在程序执行过程中检测监视点表达式是否发生变化，它是监视点机制真正生效的核心逻辑。函数从活动链表头开始遍历所有节点，并且只对类型为 WP_TYPE_WATCHPOINT 的对象进行检查，因为断点不属于表达式值变化的监视范围。对于每个监视点，函数都会再次调用 expr(cur->expr, &success) 对保存的表达式进行求值，并把当前结果与 last_value 中记录的旧值进行比较。如果表达式求值失败，说明表达式在当前状态下无法正确计算，函数会输出错误信息并将 stop 置为 true，表示调试器应当暂停执行。如果新值与旧值不同，说明被监视的表达式确实发生了变化，此时函数会打印触发信息、旧值和新值，并更新 last_value 为当前值，同时设置停止标志。最后，函数返回 stop，由上层决定是否暂停程序。

Listing 24: check_bp

```

1  bool check_bp(void) {
2      if (skip_current_bp && cpu.eip == skip_bp_addr) {
3          skip_current_bp = false;
4          return false;
5      }
6      skip_current_bp = false;
7
8      for (WP *cur = head; cur != NULL; cur = cur->next) {

```



```

9      if (cur->type == WP_TYPE_BREAKPOINT && cpu.eip == cur->addr) {
10          printf("Hit breakpoint %d at 0x%08x\n", cur->NO, cur->addr);
11          skip_current_bp = true;
12          skip_bp_addr = cur->addr;
13          return true;
14      }
15  }
16
17  return false;
18  }
19

```

check_bp 函数负责在程序执行前检测当前指令地址是否命中断点。函数首先检查 skip_current_bp 和 skip_bp_addr 这两个控制变量，这一段逻辑的作用是防止某些情况下断点在同一地址被重复触发。如果当前正处于需要跳过的断点地址，则函数会清除跳过标志并直接返回 false，表示本次不应暂停。接着，函数会把 skip_current_bp 重新置为 false，以便下一次检查正常进行。随后，函数遍历活动链表中的所有对象，只对类型为 WP_TYPE_BREAKPOINT 的节点进行比较，判断当前 cpu.eip 是否等于断点保存的地址 addr。如果命中断点，函数会打印命中信息，记录当前地址到 skip_bp_addr，并把 skip_current_bp 置为 true，然后返回 true 表示需要暂停程序。

完善后，对实现的监视点功能进行测试，结果如下：

```

wyn@ubuntu:~/Documents/ics2017/nemu$ make run
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 03:22:52, Apr 23 2026
For help, type "help"
(nemu) w $eip==0x100000
[src/monitor/debug/expr.c,94,make_token] match rules[2] = "\[a-zA-Z\]+" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[1] = "[xX][0-9a-fA-F]+" at position 6 with len 8: 0x100000
Watchpoint 0: $eip==0x100000 = 0x00000001
(nemu) w $eip==0x100005
[src/monitor/debug/expr.c,94,make_token] match rules[2] = "\[a-zA-Z\]+" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[1] = "[xX][0-9a-fA-F]+" at position 6 with len 8: 0x100005
Watchpoint 1: $eip==0x100005 = 0x00000000
(nemu) info w
NO  TYPE      VALUE      EXPR
1   watchpoint 0x00000000 $eip==0x100005
0   watchpoint 0x00000001 $eip==0x100000
(nemu) c
[src/monitor/debug/expr.c,94,make_token] match rules[2] = "\[a-zA-Z\]+" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[1] = "[xX][0-9a-fA-F]+" at position 6 with len 8: 0x100005
Watchpoint 1 triggered: $eip==0x100005
Old value = 0x00000000
New value = 0x00000001
[src/monitor/debug/expr.c,94,make_token] match rules[2] = "\[a-zA-Z\]+" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[1] = "[xX][0-9a-fA-F]+" at position 6 with len 8: 0x100000
Watchpoint 0 triggered: $eip==0x100000
Old value = 0x00000001
New value = 0x00000000
(nemu) info w
NO  TYPE      VALUE      EXPR
1   watchpoint 0x00000001 $eip==0x100005
0   watchpoint 0x00000000 $eip==0x100000
(nemu) c
[src/monitor/debug/expr.c,94,make_token] match rules[2] = "\[a-zA-Z\]+" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[1] = "[xX][0-9a-fA-F]+" at position 6 with len 8: 0x100005
Watchpoint 1 triggered: $eip==0x100005
Old value = 0x00000001
New value = 0x00000000
[src/monitor/debug/expr.c,94,make_token] match rules[2] = "\[a-zA-Z\]+" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[1] = "[xX][0-9a-fA-F]+" at position 6 with len 8: 0x100000

```

```

(nemu) info w
NO  TYPE      VALUE      EXPR
1   watchpoint 0x00000000 $eip==0x100005
0   watchpoint 0x00000000 $eip==0x100000
(nemu) c
[src/monitor/debug/expr.c,94,make_token] match rules[2] = "\[a-zA-Z\]+" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[1] = "0[xX][0-9a-fA-F]+" at position 6 with len 8: 0x100005
[src/monitor/debug/expr.c,94,make_token] match rules[2] = "\[a-zA-Z\]+" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[1] = "0[xX][0-9a-fA-F]+" at position 6 with len 8: 0x100000
[src/monitor/debug/expr.c,94,make_token] match rules[2] = "\[a-zA-Z\]+" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[1] = "0[xX][0-9a-fA-F]+" at position 6 with len 8: 0x100005
[src/monitor/debug/expr.c,94,make_token] match rules[2] = "\[a-zA-Z\]+" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[1] = "0[xX][0-9a-fA-F]+" at position 6 with len 8: 0x100000
[src/monitor/debug/expr.c,94,make_token] match rules[2] = "\[a-zA-Z\]+" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[1] = "0[xX][0-9a-fA-F]+" at position 6 with len 8: 0x100005
[src/monitor/debug/expr.c,94,make_token] match rules[2] = "\[a-zA-Z\]+" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[1] = "0[xX][0-9a-fA-F]+" at position 6 with len 8: 0x100000
[src/monitor/debug/expr.c,94,make_token] match rules[2] = "\[a-zA-Z\]+" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[1] = "0[xX][0-9a-fA-F]+" at position 6 with len 8: 0x100005
[src/monitor/debug/expr.c,94,make_token] match rules[2] = "\[a-zA-Z\]+" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,94,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,94,make_token] match rules[1] = "0[xX][0-9a-fA-F]+" at position 6 with len 8: 0x100000
nemu: HIT GOOD TRAP at eip = 0x00100026

(nemu) info w
NO  TYPE      VALUE      EXPR
1   watchpoint 0x00000000 $eip==0x100005
0   watchpoint 0x00000000 $eip==0x100000
(nemu) d 1
Deleted watchpoint/breakpoint 1
(nemu) info w
NO  TYPE      VALUE      EXPR
0   watchpoint 0x00000000 $eip==0x100000
(nemu) d 0
Deleted watchpoint/breakpoint 0
(nemu) info w
No watchpoints or breakpoints.
(nemu) q

```

5 思考题与必答题

5.1 思考题

究竟要执行多久？ 在 `cmd_c()` 函数中，调用 `cpu_exec()` 的时候传入了参数 -1，你知道这是什么意思吗？

答：这里的 -1 实际上是表示尽可能一直执行下去。因为 `cpu_exec` 的参数类型是 `uint64_t`，把 -1 传入无符号类型时会按模 2^{64} 转换，得到该类型能表示的最大值，因此效果就等价于“执行一个极大的指令数”，直到程序结束或遇到断点、监视点等停止条件为止。

谁来指示程序的结束？ 在程序设计课上老师告诉你，当程序执行到 `main()` 函数返回处的时候，程序就退出了，你对此深信不疑。但你是否怀疑过，凭什么程序执行到 `main()` 函数的返回处就结束了？如果有人告诉你，程序设计课上老师的说法是错的，你有办法来证明/反驳吗？如果你对此感兴趣，请在互联网上搜索相关内容。

答：程序并不是在 `main()` 的返回位置“自动结束”的，而是由 C 运行时启动代码和操作系统共同完成终止流程：main 返回后，运行时会把返回值交给退出流程，最终由 `exit` 或系统级终止接口结束进程。Microsoft 的文档也明确写到，程序通常在从 `main` 返回或到达 `main` 末尾时停止执行，但真正

负责终止进程的是运行时/退出机制,而不是 main 本身。main 只是把控制权交回给运行时,运行时再调用 exit,由它完成进程终止。

温故而知新 框架代码中定义 wp_pool 等变量的时候使用了关键字 static,static 在此处的含义是什么?为什么要在此处使用它?

答:这里的 static 表示这些变量具有静态存储期,并且在文件作用域下还具有内部链接属性。静态存储期意味着变量在程序整个运行期间都存在,不会因为函数返回而销毁;内部链接则意味着这些变量只在当前源文件内可见,其他源文件无法直接访问。把 wp_pool、head、free_ 这类调试器内部管理变量设为 static,可以避免与其他文件中的同名符号冲突,同时把监视点池、链表头和空闲链表等实现细节封装在 watchpoint.c 内部,不暴露给外部模块。这样既能保证数据在整个调试过程中持续有效,又能增强模块封装性。

一点也不能长? 我们知道 int3 指令不带任何操作数,操作码为 1 个字节,因此指令的长度是 1 个字节,这是必须的吗?假设有种 x86 体系结构的变种 my-x86,除了 int3 指令的长度变成了 2 个字节之外,其余指令和 x86 相同。在 my-x86 中,文章中的断点机制还可以正常工作吗?为什么?

答:在 x86 上,INT3 之所以适合做软件断点,一个关键原因就是它是 1 字节指令,正好可以替换目标指令的第一个字节而不破坏后续代码。GDB 的内部文档也说明了,软件断点通常通过把程序指令替换成 trap 指令实现,而且这种断点指令应当尽可能短,最好不超过体系结构中最短指令的长度。基于这个原理,如果在假设的 my-x86 中把 int3 改成 2 字节,而其他指令仍保持原样,那么原有的断点机制就不再可靠:因为插入断点时必须覆盖 2 个字节,可能把原始指令的更多部分一起破坏,甚至覆盖到相邻指令,导致程序流被破坏。也就是说,这种机制在“断点指令必须足够短”这一前提下才成立;一旦 int3 变长,原来的实现思路就会失效。

随心所欲的断点 如果把断点设置在指令的非首字节(中间或末尾),会发生什么?你可以在 GDB 中尝试一下,然后思考并解释其中的缘由。

答:如果断点被放在某条指令的中间或末尾,那么它就不再落在合法的指令边界上,CPU 取指和解码时会把这 1 个字节当成新的操作码或操作数的一部分,从而破坏原指令的完整编码。若强行把断点放在非首字节,通常会导致指令流被破坏,轻则执行出错、触发非法指令异常,重则程序行为完全失真。

NEMU 的前世今生 你已经对 NEMU 的工作方式有所了解了。事实上在 NEMU 诞生之前,NEMU 曾经有一段时间并不叫 NEMU,而是叫 NDB (NJU Debugger),后来由于某种原因才改名为 NEMU。如果你想知道这一段史前的秘密,你首先需要了解这样个问题:模拟器 (Emulator) 和调试器 (Debugger) 有什么不同?更具体地,和 NEMU 相比,GDB 到底是如何调试程序的?

答:两者的核心差别在于:模拟器是“把机器本身在软件中重新实现出来”,而调试器是“控制一个已经在运行的程序”。模拟器会逐条模拟指令执行、寄存器状态、内存访问乃至设备行为;而 GDB 这类调试器并不模拟整机,它通常通过 ptrace 等机制附着到真实进程上,再用软件断点、单步执行和状态读取来控制程序。GDB 的断点实现原理就是把原指令替换成 trap,等程序停下后再恢复原指令并单步执行,这说明它是在“观察和操控进程”,不是“自己模拟 CPU”。相比之下,NEMU 通过解释执行每条指令来模拟完整的 CPU/内存/设备环境,所以它属于模拟器;GDB 则属于调试器。也正因为这种区别,NEMU 更像一个“被软件实现出来的机器”,而 GDB 更像一个“依附在机器上的观察与控制工具”。

5.2 必答题

1 查阅 i386 手册理解了科学查阅手册的方法之后, 请你尝试在 i386 手册中查阅以下问题所在的位置, 把需要阅读的范围写到你的实验报告里面:

- EFLAGS 寄存器中的 CF 位是什么意思?
- ModR/M 字节是什么?
- mov 指令的具体格式是怎么样的?

答: 通过查阅 i386.pdf 可以看到, i386 手册采用“先总体介绍体系结构, 再在指令参考中逐条列出 opcode、Instruction、Operation 和 Description”的组织方式。查找具体问题时, 可以先从目录或 PDF 文本搜索关键字定位章节, 再阅读该章节前后的表格和说明。

1. EFLAGS 寄存器中的 CF 位是什么意思?

需要阅读的范围:

- Chapter 2, 2.3.4 EFLAGS Register, 重点看 Figure 2-8. EFLAGS Register 和 2.3.4.1 Status Flags。
- Appendix C, Status Flags' Functions。

阅读可知, EFLAGS 是 32 位标志寄存器, 低 16 位兼容 8086/80286 的 FLAGS。CF 是 bit 0, 名称为 Carry Flag。Appendix C 中说明: CF 在最高位发生进位或借位时置 1, 否则清 0。因此它主要反映无符号加减运算中的进位/借位, 也会被移位、循环移位等指令用来保存被移出的 bit。

2.ModR/M 字节是什么?

需要阅读的范围:

- Chapter 17, 17.2 Instruction Format, 先看 opcode 表中 /digit、/r 的说明。
- Chapter 17, 17.2.1 ModR/M and SIB Bytes, 重点看 Figure 17-2. ModR/M and SIB Byte Formats, 以及 Table 17-2、Table 17-3、Table 17-4。

阅读可知, ModR/M 字节位于许多指令的 opcode 字节之后, 用来补充说明操作数位置和寻址方式。它由三个字段组成: mod 占最高 2 位, 和 r/m 组合后描述寄存器或内存寻址形式; reg 占中间 3 位, 可以表示一个寄存器编号, 也可以作为 opcode 扩展; r/m 占最低 3 位, 可以表示寄存器操作数, 也可以和 mod 一起形成内存寻址方式。在 32 位复杂寻址中, 某些 ModR/M 编码还会引出后续的 SIB 字节。

3.mov 指令的具体格式是怎么样的?

需要阅读的范围:

- Instruction Set Reference 中 MOV – Move Data 条目。
- 同时需要结合 Chapter 17 的 17.2.2.1 Opcode、17.2.2.2 Instruction 对 opcode 表中 /r、+rb/+rw/+rd、ib/iw/id、moffs 等记号的解释。

MOV 指令的 Operation 是: DEST <- SRC; 手册中列出的主要格式包括:

1	88 /r	MOV r/m8,r8
2	89 /r	MOV r/m16,r16
3	89 /r	MOV r/m32,r32
4	8A /r	MOV r8,r/m8

```

5      8B /r      MOV r16,r/m16
6      8B /r      MOV r32,r/m32
7      8C /r      MOV r/m16,Sreg
8      8E /r      MOV Sreg,r/m16
9      A0         MOV AL,moffs8
10     A1         MOV AX,moffs16
11     A1         MOV EAX,moffs32
12     A2         MOV moffs8,AL
13     A3         MOV moffs16,AX
14     A3         MOV moffs32,EAX
15     B0 + rb     MOV reg8,imm8
16     B8 + rw     MOV reg16,imm16
17     B8 + rd     MOV reg32,imm32
18     C6 /0      MOV r/m8,imm8
19     C7 /0      MOV r/m16,imm16
20     C7 /0      MOV r/m32,imm32
21

```

其中 /r 表示该指令后面带 ModR/M 字节,ModR/M 同时给出寄存器操作数和 r/m 操作数;+rb/+rw/+rd 表示寄存器编号直接编码在 opcode 的低 3 位;imm8/imm16/imm32 是立即数;moffs8/moffs16/moffs32 是直接内存偏移。

2 shell 命令完成 PA1 的内容之后,nemu/目录下的所有.c 和.h 文件总共有多少行代码? 你是使用什么命令得到这个结果的? 和框架代码相比, 你在 PA1 中编写了多少少行代码?(Hint: 目前 2017 分支中记录的正好是做 PA 之前的状态, 思考一下应该如何回到”过去”?) 你可以把这条命令写入 Makefile 中, 随着实验进度的推进, 你可以很方便地统计工程的代码行数, 例如敲入 make count: 就会自动运行统计代码行数的命令, 再来个难一点的, 除去空行之外,nemu/目录下的所有.c 和.h 文件总共有多少行代码?

答: 使用 “git checkout 分支名” 切换到对应分支后, 使用如下命令可以统计 nemu/目录下所有.c 和.h 文件的行数以及非空行数:

```

1      //行数
2      find . -type f \( -name "*.c" -o -name "*.h" \) | xargs wc -l
3      //非空行数
4      find . -type f \( -name "*.c" -o -name "*.h" \) | xargs grep -hve '^[[:space:]]*$' | wc -l
5

```

很可惜的是, 在书写该报告时, 我已将 pa1 分支合并入 master 分支, 对比不出所增行数, 这里用 pa2 相比 pa1 演示如何统计所编写代码行数:


```
wyn@ubuntu:~/Documents/ics2017/nemu$ find . -type f \( -name "*.c" -o -name "*.h" \) | xargs wc -l
14 ./include/device/mmio.h
13 ./include/device/port-io.h
18 ./include/memory/memory.h
82 ./include/memory/mmu.h
13 ./include/macro.h
29 ./include/monitor/watchpoint.h
7 ./include/monitor/monitor.h
8 ./include/monitor/expr.h
45 ./include/debug.h
29 ./include/common.h
115 ./include/cpu/decode.h
53 ./include/cpu/exec.h
64 ./include/cpu/reg.h
139 ./include/cpu/rtl.h
8 ./include/nemu.h
28 ./src/device/timer.c
38 ./src/device/vga.c
98 ./src/device/device.c
70 ./src/device/io/mmio.c
56 ./src/device/io/port-io.c
70 ./src/device/keyboard.c
29 ./src/device/serial.c
28 ./src/memory/memory.c
184 ./src/monitor/debug/watchpoint.c
308 ./src/monitor/debug/expr.c
296 ./src/monitor/debug/ui.c
144 ./src/monitor/monitor.c
56 ./src/monitor/cpu-exec.c
106 ./src/monitor/diff-test/gdb-host.c
48 ./src/monitor/diff-test/protocol.h
157 ./src/monitor/diff-test/diff-test.c
314 ./src/monitor/diff-test/protocol.c
31 ./src/misc/logo.c
65 ./src/cpu/exec/system.c
223 ./src/cpu/exec/arith.c
60 ./src/cpu/exec/logic.c
77 ./src/cpu/exec/data-mov.c
43 ./src/cpu/exec/control.c
255 ./src/cpu/exec/exec.c
9 ./src/cpu/exec/prefix.c
46 ./src/cpu/exec/special.c
8 ./src/cpu/exec/all-instr.h
32 ./src/cpu/exec/cc.c
311 ./src/cpu/decode/decode.c
113 ./src/cpu/decode/modrm.c
43 ./src/cpu/reg.c
13 ./src/cpu/intr.c
12 ./src/main.c
4057 total
wyn@ubuntu:~/Documents/ics2017/nemu$ find . -type f \( -name "*.c" -o -name "*.h" \) | xargs grep -hve '^[[:space:]]*$' | wc -l
3314

wyn@ubuntu:~/Documents/ics2017/nemu$ find . -type f \( -name "*.c" -o -name "*.h" \) | xargs wc -l
14 ./include/device/mmio.h
13 ./include/device/port-io.h
18 ./include/memory/memory.h
82 ./include/memory/mmu.h
13 ./include/macro.h
29 ./include/monitor/watchpoint.h
7 ./include/monitor/monitor.h
8 ./include/monitor/expr.h
45 ./include/debug.h
29 ./include/common.h
115 ./include/cpu/decode.h
53 ./include/cpu/exec.h
83 ./include/cpu/reg.h
280 ./include/cpu/rtl.h
8 ./include/nemu.h
28 ./src/device/timer.c
38 ./src/device/vga.c
98 ./src/device/device.c
70 ./src/device/io/mmio.c
56 ./src/device/io/port-io.c
70 ./src/device/keyboard.c
29 ./src/device/serial.c
38 ./src/memory/memory.c
184 ./src/monitor/debug/watchpoint.c
308 ./src/monitor/debug/expr.c
296 ./src/monitor/debug/ui.c
144 ./src/monitor/monitor.c
56 ./src/monitor/cpu-exec.c
106 ./src/monitor/diff-test/gdb-host.c
48 ./src/monitor/diff-test/protocol.h
181 ./src/monitor/diff-test/diff-test.c
314 ./src/monitor/diff-test/protocol.c
31 ./src/misc/logo.c
69 ./src/cpu/exec/system.c
310 ./src/cpu/exec/arith.c
154 ./src/cpu/exec/logic.c
96 ./src/cpu/exec/data-mov.c
51 ./src/cpu/exec/control.c
255 ./src/cpu/exec/exec.c
9 ./src/cpu/exec/prefix.c
46 ./src/cpu/exec/special.c
52 ./src/cpu/exec/all-instr.h
54 ./src/cpu/exec/cc.c
315 ./src/cpu/decode/decode.c
113 ./src/cpu/decode/modrm.c
43 ./src/cpu/reg.c
13 ./src/cpu/intr.c
12 ./src/main.c
4404 total
wyn@ubuntu:~/Documents/ics2017/nemu$ find . -type f \( -name "*.c" -o -name "*.h" \) | xargs grep -hve '^[[:space:]]*$' | wc -l
3620
```

上图左侧为 pa1 分支下统计，可见此时有 4057 行，其中非空行 3314；右侧为 pa2 分支下统计，可见此时有 4404 行，其中非空行 3620。做差可知，pa2 任务中我编写了 347 行，非空行 306。

在 nemu/Makefile 中添加以下内容：

Listing 25: make count

```
1 count:
2 @echo "Total lines of .c/.h files:"
3 @find . -type f \( -name "*.c" -o -name "*.h" \) | xargs wc -l
4 @echo "Nonblank lines of .c/.h files:"
5 @find . -type f \( -name "*.c" -o -name "*.h" \) | xargs grep -hve '^[[:space:]]*$' | wc -l
6
```

此时就可以使用 make count 命令来快捷统计 nemu 目录下所有.c 和.h 文件的行数以及非空行数，如下图所示：

```
wyn@ubuntu:~/Documents/ics2017/nemu$ make count
Total lines of ./h files:
14 ./include/device/mmio.h
13 ./include/device/port-io.h
18 ./include/memory/memory.h
82 ./include/memory/mmu.h
13 ./include/macro.h
29 ./include/monitor/watchpoint.h
7 ./include/monitor/monitor.h
8 ./include/monitor/expr.h
45 ./include/debug.h
29 ./include/common.h
115 ./include/cpu/decode.h
53 ./include/cpu/exec.h
83 ./include/cpu/reg.h
200 ./include/cpu/rtl.h
8 ./include/nemu.h
28 ./src/device/timer.c
38 ./src/device/vga.c
98 ./src/device/device.c
70 ./src/device/io/mmio.c
56 ./src/device/io/port-io.c
70 ./src/device/keyboard.c
29 ./src/device/serial.c
38 ./src/memory/memory.c
184 ./src/monitor/debug/watchpoint.c
308 ./src/monitor/debug/expr.c
296 ./src/monitor/debug/ui.c
144 ./src/monitor/monitor.c
56 ./src/monitor/cpu-exec.c
106 ./src/monitor/diff-test/gdb-host.c
48 ./src/monitor/diff-test/protocol.h
181 ./src/monitor/diff-test/diff-test.c
314 ./src/monitor/diff-test/protocol.c
31 ./src/misc/logo.c
69 ./src/cpu/exec/system.c
310 ./src/cpu/exec/arith.c
154 ./src/cpu/exec/logic.c
96 ./src/cpu/exec/data-mov.c
51 ./src/cpu/exec/control.c
255 ./src/cpu/exec/exec.c
9 ./src/cpu/exec/prefix.c
46 ./src/cpu/exec/special.c
52 ./src/cpu/exec/all-instr.h
54 ./src/cpu/exec/cc.c
315 ./src/cpu/decode/decode.c
113 ./src/cpu/decode/modrm.c
43 ./src/cpu/reg.c
13 ./src/cpu/intr.c
12 ./src/main.c
4404 total
Nonblank lines of ./h files:
3620
```

3 使用 man 打开工程目录下的 Makefile 文件, 你会在 CFLAGS 变量中看到 gcc 的一些编译选项. 请解释 gcc 中的 -Wall 和 -Werror 有什么作用? 为什么要使用 -Wall 和 -Werror 有什么作用? 为什么要使用 -Wall 和 -Werror?

答: nemu/Makefile 中的 CFLAGS 包含:

```
1 CFLAGS += -O2 -MMD -Wall -Werror -ggdb $(INCLUDES)
2
```

-Wall 的作用是打开 GCC 中一组常用且重要的警告, 例如未使用变量、可疑的控制流、函数声明或类型相关问题等。它并不是打开所有警告, 而是打开一批 GCC 认为通常值得检查的警告。

-Werror 的作用是把警告当作错误处理。也就是说, 只要编译过程中出现 warning, 编译就会失败。

实验中使用 -Wall -Werror 的原因是: PA 实验实现的是模拟器底层代码, 很多 warning 都可能对应真实 bug, 例如类型截断、未初始化变量、函数返回值不匹配、格式化字符串错误等。如果只开启 warning 而允许继续编译, 问题可能被忽略, 最终表现为运行时错误或难以定位的模拟器行为错误。使用 -Werror 可以强制在问题刚出现时修复它, 保证提交的代码没有已知编译警告。